- ROS 2 estimator
- logical tether measurements

After that works, improve realism step by step.

Because your real research value is not “making a cable in Gazebo.”
Your real value is **estimating the submarine global position using buoy GPS + tether geometry**.

If you want, next I can give you a **full ROS 2 + Gazebo project setup plan with package names, launch flow, topics, and a sample estimator node**.

Sources

i have geo model , gazebo aand ros2 i need step by step instructions to do semulations , with full codes instruction

Thought for 1m 41s

Use this as your **minimum working FYP simulation**:

- **Gazebo Sim** = visual world with a buoy at the surface and a submarine underwater
- **ROS 2** = estimator side
- **Tether** = logical tether first, not full cable physics
- **Buoy GPS** = simulated from buoy pose
- **Submarine actual GPS** = estimated by your geo model

This approach is the safest first step. Gazebo already supports ROS 2 integration through `ros_gz_bridge`, and Gazebo’s underwater stack includes buoyancy / hydrodynamics support if you want to make it more realistic later. For a stable first build, use ROS 2 + Gazebo’s recommended pairing for your distro; for example, ROS 2 Humble works with Gazebo Fortress, and ROS 2 Jazzy works with Gazebo Harmonic. The default install path is `sudo apt-get install ros-${ROS_DISTRO}-ros-gz`. gazebosim.org +2

1) What you will build

You will create:

1. a Gazebo world called `ocean_world`
2. a **buoy model** at the water surface
3. a **submarine model** underwater
4. Gazebo **OdometryPublisher** systems on both models so Gazebo publishes odometry on `/model/{name}/odometry` by default, which can be bridged to ROS 2 as `nav_msgs/msg/Odometry` using `ros_gz_bridge` gazebosim.org +1
5. a ROS 2 node that converts buoy local pose to fake RTK GPS
6. a ROS 2 node that computes submarine estimated GPS using your geo model
7. a ROS 2 node that compares estimated position vs ground truth

You will also use Gazebo's `/world/<world>/set_pose` service, provided by the `UserCommands` system, to move the submarine during testing. gazebosim.org +1

2) Folder structure

Create a workspace:

Bash

```bash
mkdir -p ~/fyp_ws/src
cd ~/fyp_ws/src
ros2 pkg create --build-type ament_python buoy_sub_geo_sim
```

After that, make your package look like this:

Plain text

```
buoy_sub_geo_sim/
├── buoy_sub_geo_sim/
│   ├── __init__.py
│   ├── fake_buoy_gps.py
│   ├── geo_estimator.py
│   └── evaluator.py
├── launch/
│   └── sim.launch.py
├── worlds/
│   └── ocean_world.sdf
├── package.xml
```

```
├── setup.py
└── setup.cfg
```

3) package.xml

Replace with this:

XML

```xml
<?xml version="1.0"?>
<package format="3">
  <name>buoy_sub_geo_sim</name>
  <version>0.0.1</version>
  <description>Buoy + submarine + geo model simulation in Gazebo and ROS 2

  <maintainer email="you@example.com">you</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>nav_msgs</exec_depend>
  <exec_depend>sensor_msgs</exec_depend>
  <exec_depend>std_msgs</exec_depend>
  <exec_depend>ros_gz_bridge</exec_depend>
  <exec_depend>ros_gz_sim</exec_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

4) setup.py

Python Run

```
from setuptools import setup
from glob import glob
import os

package_name = 'buoy_sub_geo_sim'

setup(
    name=package_name,
    version='0.0.1',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name] if os.path.exists('resource/' + pack
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name + '/launch', glob('launch/*.py')),
        ('share/' + package_name + '/worlds', glob('worlds/*.sdf')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='you',
    maintainer_email='you@example.com',
    description='Buoy + submarine + geo model simulation in Gazebo and ROS
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'fake_buoy_gps = buoy_sub_geo_sim.fake_buoy_gps:main',
            'geo_estimator = buoy_sub_geo_sim.geo_estimator:main',
            'evaluator = buoy_sub_geo_sim.evaluator:main',
        ],
    },
)
```

5) setup.cfg

INI

```
[develop]
script_dir=$base/lib/buoy_sub_geo_sim
[install]
install_scripts=$base/lib/buoy_sub_geo_sim
```

6) `worlds/ocean_world.sdf`

This is the first working version. It is intentionally simple: the buoy stays at the surface and the submarine starts underwater. Later you can replace the simple shapes with your real meshes and add buoyancy / hydrodynamics plugins. Gazebo system plugins such as `UserCommands`, `SceneBroadcaster`, `PosePublisher`, and `OdometryPublisher` are loaded through SDF. `OdometryPublisher` supports custom topics, 3D odometry, frame names, and noise. gazebosim.org +2

XML

```xml
<?xml version="1.0" ?>
<sdf version="1.9">
  <world name="ocean_world">

    <!-- Core world systems -->
    <plugin
      filename="gz-sim-physics-system"
      name="gz::sim::systems::Physics"/>

    <plugin
      filename="gz-sim-scene-broadcaster-system"
      name="gz::sim::systems::SceneBroadcaster"/>

    <plugin
      filename="gz-sim-user-commands-system"
      name="gz::sim::systems::UserCommands"/>

    <!-- Gravity -->
    <gravity>0 0 -9.81</gravity>

    <!-- Water surface visual -->
    <model name="water_surface">
      <static>true</static>
      <pose>0 0 0 0 0 0</pose>
      <link name="link">
        <visual name="visual">
          <geometry>
            <box>
              <size>200 200 0.02</size>
            </box>
          </geometry>
          <material>
            <ambient>0.1 0.3 0.8 0.4</ambient>
            <diffuse>0.1 0.3 0.8 0.4</diffuse>
          </material>
```

```
      </visual>
    </link>
  </model>

  <!-- Buoy -->
  <model name="buoy">
    <static>true</static>
    <pose>0 0 0.3 0 0 0</pose>

    <link name="base_link">
      <inertial>
        <mass>10.0</mass>
        <inertia>
          <ixx>1.0</ixx><iyy>1.0</iyy><izz>1.0</izz>
          <ixy>0.0</ixy><ixz>0.0</ixz><iyz>0.0</iyz>
        </inertia>
      </inertial>

      <collision name="collision">
        <geometry>
          <cylinder>
            <radius>0.25</radius>
            <length>0.6</length>
          </cylinder>
        </geometry>
      </collision>

      <visual name="visual">
        <geometry>
          <cylinder>
            <radius>0.25</radius>
            <length>0.6</length>
          </cylinder>
        </geometry>
        <material>
          <ambient>1 0.6 0 1</ambient>
          <diffuse>1 0.6 0 1</diffuse>
        </material>
      </visual>
    </link>

    <plugin
      filename="gz-sim-odometry-publisher-system"
      name="gz::sim::systems::OdometryPublisher">
      <dimensions>3</dimensions>
      <odom_publish_frequency>30</odom_publish_frequency>
      <odom_topic>/model/buoy/odometry</odom_topic>
      <odom_frame>world</odom_frame>
      <robot_base_frame>buoy/base_link</robot_base_frame>
```

```
    </plugin>
  </model>

  <!-- Submarine -->
  <model name="submarine">
    <static>true</static>
    <pose>5 0 -8 0 0 0</pose>

    <link name="base_link">
      <inertial>
        <mass>25.0</mass>
        <inertia>
          <ixx>2.0</ixx><iyy>2.0</iyy><izz>2.0</izz>
          <ixy>0.0</ixy><ixz>0.0</ixz><iyz>0.0</iyz>
        </inertia>
      </inertial>

      <collision name="collision">
        <geometry>
          <box>
            <size>1.4 0.5 0.4</size>
          </box>
        </geometry>
      </collision>

      <visual name="visual">
        <geometry>
          <box>
            <size>1.4 0.5 0.4</size>
          </box>
        </geometry>
        <material>
          <ambient>0.2 0.2 0.2 1</ambient>
          <diffuse>0.2 0.2 0.2 1</diffuse>
        </material>
      </visual>
    </link>

    <plugin
      filename="gz-sim-odometry-publisher-system"
      name="gz::sim::systems::OdometryPublisher">
      <dimensions>3</dimensions>
      <odom_publish_frequency>30</odom_publish_frequency>
      <odom_topic>/model/submarine/odometry</odom_topic>
      <odom_frame>world</odom_frame>
      <robot_base_frame>submarine/base_link</robot_base_frame>
    </plugin>
  </model>
```

```
    <!-- Sun -->
    <light name="sun" type="directional">
      <cast_shadows>true</cast_shadows>
      <pose>0 0 20 0 0 0</pose>
      <diffuse>1 1 1 1</diffuse>
      <specular>0.2 0.2 0.2 1</specular>
      <direction>-0.5 0.1 -1</direction>
    </light>

  </world>
</sdf>
```

7) buoy_sub_geo_sim/fake_buoy_gps.py

This converts the buoy's local `x,y,z` in Gazebo to fake latitude and longitude around an origin. In your real system the buoy has RTK GPS, so this node acts as that RTK source.

Python Run

```python
import math

import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry
from sensor_msgs.msg import NavSatFix


class FakeBuoyGPS(Node):
    def __init__(self):
        super().__init__('fake_buoy_gps')

        self.declare_parameter('origin_lat', 7.2906)
        self.declare_parameter('origin_lon', 80.6337)
        self.declare_parameter('origin_alt', 0.0)

        self.origin_lat = float(self.get_parameter('origin_lat').value)
        self.origin_lon = float(self.get_parameter('origin_lon').value)
        self.origin_alt = float(self.get_parameter('origin_alt').value)

        self.sub = self.create_subscription(
            Odometry,
            '/buoy/odom',
            self.odom_callback,
            10
        )
```

```
        self.pub = self.create_publisher(NavSatFix, '/buoy/rtk_fix', 10)

        self.get_logger().info('fake_buoy_gps started')

    def odom_callback(self, msg: Odometry):
        x = msg.pose.pose.position.x   # East
        y = msg.pose.pose.position.y   # North
        z = msg.pose.pose.position.z

        lat_rad = math.radians(self.origin_lat)
        meters_per_deg_lat = 111320.0
        meters_per_deg_lon = 111320.0 * math.cos(lat_rad)

        dlat = y / meters_per_deg_lat
        dlon = x / meters_per_deg_lon

        nav = NavSatFix()
        nav.header.stamp = self.get_clock().now().to_msg()
        nav.header.frame_id = 'world'
        nav.latitude = self.origin_lat + dlat
        nav.longitude = self.origin_lon + dlon
        nav.altitude = self.origin_alt + z

        # Small covariance for "RTK-like" simulation
        nav.position_covariance = [
            0.01, 0.0, 0.0,
            0.0, 0.01, 0.0,
            0.0, 0.0, 0.04
        ]
        nav.position_covariance_type = NavSatFix.COVARIANCE_TYPE_DIAGONAL_

        self.pub.publish(nav)


def main():
    rclpy.init()
    node = FakeBuoyGPS()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

8) buoy_sub_geo_sim/geo_estimator.py

This is the main estimator.

Right now it includes a **straight-tether example model**.

Replace the part inside `run_geo_model()` with your own geo model.

It subscribes to:

- buoy RTK GPS
- buoy odom
- submarine odom truth

It publishes:

- estimated submarine GPS
- estimated submarine local odom

Python Run

```python
import math

import rclpy
from rclpy.node import Node

from nav_msgs.msg import Odometry
from sensor_msgs.msg import NavSatFix
from geometry_msgs.msg import Quaternion


def yaw_from_quaternion(q: Quaternion) -> float:
    siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
    cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
    return math.atan2(siny_cosp, cosy_cosp)


class GeoEstimator(Node):
    def __init__(self):
        super().__init__('geo_estimator')

        self.declare_parameter('origin_lat', 7.2906)
        self.declare_parameter('origin_lon', 80.6337)
        self.declare_parameter('origin_alt', 0.0)

        self.origin_lat = float(self.get_parameter('origin_lat').value)
        self.origin_lon = float(self.get_parameter('origin_lon').value)
        self.origin_alt = float(self.get_parameter('origin_alt').value)

        self.buoy_fix = None
```

```
        self.buoy_odom = None
        self.sub_odom = None

        self.create_subscription(NavSatFix, '/buoy/rtk_fix', self.on_buoy_
        self.create_subscription(Odometry, '/buoy/odom', self.on_buoy_odom
        self.create_subscription(Odometry, '/submarine/odom', self.on_sub_

        self.est_fix_pub = self.create_publisher(NavSatFix, '/submarine/es
        self.est_odom_pub = self.create_publisher(Odometry, '/submarine/es

        self.timer = self.create_timer(0.1, self.update)

        self.get_logger().info('geo_estimator started')

    def on_buoy_fix(self, msg):
        self.buoy_fix = msg

    def on_buoy_odom(self, msg):
        self.buoy_odom = msg

    def on_sub_odom(self, msg):
        self.sub_odom = msg

    def run_geo_model(self, buoy_x, buoy_y, buoy_z, buoy_yaw, sub_x, sub_y
        """
        Replace this with YOUR geo model.
        Current version uses a simple straight-tether geometry example.

        Returns estimated submarine local coordinates (x, y, z)
        """

        dx = sub_x - buoy_x
        dy = sub_y - buoy_y
        dz = sub_z - buoy_z

        tether_length = math.sqrt(dx*dx + dy*dy + dz*dz)
        horizontal = math.sqrt(dx*dx + dy*dy)

        azimuth_world = math.atan2(dy, dx)
        azimuth_relative_to_buoy = azimuth_world - buoy_yaw
        depth = buoy_z - sub_z

        # --- Example: reconstruct local position from geometry ---
        est_horizontal = math.sqrt(max(tether_length**2 - depth**2, 0.0))
        est_azimuth_world = buoy_yaw + azimuth_relative_to_buoy

        est_dx = est_horizontal * math.cos(est_azimuth_world)
        est_dy = est_horizontal * math.sin(est_azimuth_world)
        est_dz = -depth
```

```
        est_x = buoy_x + est_dx
        est_y = buoy_y + est_dy
        est_z = buoy_z + est_dz

        return est_x, est_y, est_z

    def local_xy_to_latlon(self, x, y, z):
        lat_rad = math.radians(self.origin_lat)
        meters_per_deg_lat = 111320.0
        meters_per_deg_lon = 111320.0 * math.cos(lat_rad)

        lat = self.origin_lat + (y / meters_per_deg_lat)
        lon = self.origin_lon + (x / meters_per_deg_lon)
        alt = self.origin_alt + z
        return lat, lon, alt

    def update(self):
        if self.buoy_fix is None or self.buoy_odom is None or self.sub_odo
            return

        buoy_pos = self.buoy_odom.pose.pose.position
        buoy_ori = self.buoy_odom.pose.pose.orientation
        sub_pos = self.sub_odom.pose.pose.position

        buoy_yaw = yaw_from_quaternion(buoy_ori)

        est_x, est_y, est_z = self.run_geo_model(
            buoy_pos.x, buoy_pos.y, buoy_pos.z,
            buoy_yaw,
            sub_pos.x, sub_pos.y, sub_pos.z
        )

        lat, lon, alt = self.local_xy_to_latlon(est_x, est_y, est_z)

        est_fix = NavSatFix()
        est_fix.header.stamp = self.get_clock().now().to_msg()
        est_fix.header.frame_id = 'world'
        est_fix.latitude = lat
        est_fix.longitude = lon
        est_fix.altitude = alt
        self.est_fix_pub.publish(est_fix)

        est_odom = Odometry()
        est_odom.header.stamp = est_fix.header.stamp
        est_odom.header.frame_id = 'world'
        est_odom.child_frame_id = 'submarine_estimated'
        est_odom.pose.pose.position.x = est_x
        est_odom.pose.pose.position.y = est_y
```

```
        est_odom.pose.pose.position.z = est_z
        est_odom.pose.pose.orientation.w = 1.0
        self.est_odom_pub.publish(est_odom)


def main():
    rclpy.init()
    node = GeoEstimator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

9) buoy_sub_geo_sim/evaluator.py

This gives you estimation error in meters, which is useful for your thesis results.

Python Run

```
import math

import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry


class Evaluator(Node):
    def __init__(self):
        super().__init__('evaluator')

        self.truth = None
        self.estimate = None

        self.create_subscription(Odometry, '/submarine/odom', self.truth_c
        self.create_subscription(Odometry, '/submarine/estimated_odom', se

        self.timer = self.create_timer(0.5, self.report)

        self.get_logger().info('evaluator started')

    def truth_cb(self, msg):
        self.truth = msg

    def est_cb(self, msg):
        self.estimate = msg
```

```
    def report(self):
        if self.truth is None or self.estimate is None:
            return

        tx = self.truth.pose.pose.position.x
        ty = self.truth.pose.pose.position.y
        tz = self.truth.pose.pose.position.z

        ex = self.estimate.pose.pose.position.x
        ey = self.estimate.pose.pose.position.y
        ez = self.estimate.pose.pose.position.z

        err = math.sqrt((tx - ex) ** 2 + (ty - ey) ** 2 + (tz - ez) ** 2)

        self.get_logger().info(f'Position error = {err:.3f} m')


def main():
    rclpy.init()
    node = Evaluator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

10) `launch/sim.launch.py`

This launches Gazebo, bridges the odometry topics, and starts your nodes.

`ros_gz_bridge` can bridge `nav_msgs/msg/Odometry` with `gz.msgs.Odometry`, which is exactly what `OdometryPublisher` emits. docs.ros.org +1

Python Run

```python
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import ExecuteProcess
from launch_ros.actions import Node


def generate_launch_description():
    pkg_share = get_package_share_directory('buoy_sub_geo_sim')
    world = os.path.join(pkg_share, 'worlds', 'ocean_world.sdf')
```

```
    gazebo = ExecuteProcess(
        cmd=['gz', 'sim', '-r', world],
        output='screen'
    )

    bridge = Node(
        package='ros_gz_bridge',
        executable='parameter_bridge',
        arguments=[
            '/model/buoy/odometry@nav_msgs/msg/Odometry[gz.msgs.Odometry',
            '/model/submarine/odometry@nav_msgs/msg/Odometry[gz.msgs.Odome
        ],
        output='screen',
        remappings=[
            ('/model/buoy/odometry', '/buoy/odom'),
            ('/model/submarine/odometry', '/submarine/odom'),
        ]
    )

    fake_gps = Node(
        package='buoy_sub_geo_sim',
        executable='fake_buoy_gps',
        name='fake_buoy_gps',
        parameters=[{
            'origin_lat': 7.2906,
            'origin_lon': 80.6337,
            'origin_alt': 0.0,
        }],
        output='screen'
    )

    estimator = Node(
        package='buoy_sub_geo_sim',
        executable='geo_estimator',
        name='geo_estimator',
        parameters=[{
            'origin_lat': 7.2906,
            'origin_lon': 80.6337,
            'origin_alt': 0.0,
        }],
        output='screen'
    )

    evaluator = Node(
        package='buoy_sub_geo_sim',
        executable='evaluator',
        name='evaluator',
        output='screen'
```

```
    )

    return LaunchDescription([
        gazebo,
        bridge,
        fake_gps,
        estimator,
        evaluator,
    ])
```

11) Build

From workspace root:

Bash

```bash
cd ~/fyp_ws
colcon build
source install/setup.bash
```

12) Run the simulation

Bash

```bash
ros2 launch buoy_sub_geo_sim sim.launch.py
```

You should now have:

- Gazebo world open
- buoy visible at surface
- submarine visible underwater
- ROS 2 topics running

Check topics:

Bash

```bash
ros2 topic list
```

You should see:

Plain text

```
/buoy/odom
/buoy/rtk_fix
/submarine/odom
/submarine/estimated_fix
/submarine/estimated_odom
```

13) Test the simulation by moving the submarine

Gazebo's `UserCommands` system exposes `/world/ocean_world/set_pose`, which lets you reposition a model during runtime. gazebosim.org +1

Open a new terminal:

Bash

```bash
source ~/fyp_ws/install/setup.bash
```

Move the submarine:

Bash

```bash
gz service -s /world/ocean_world/set_pose \
  --reqtype gz.msgs.Pose \
  --reptype gz.msgs.Boolean \
  --timeout 1000 \
  --req 'name: "submarine", position: {x: 8, y: 2, z: -10}, orientation: {
```

Try again:

Bash

```bash
gz service -s /world/ocean_world/set_pose \
  --reqtype gz.msgs.Pose \
  --reptype gz.msgs.Boolean \
  --timeout 1000 \
  --req 'name: "submarine", position: {x: 12, y: -4, z: -14}, orientation:
```

Now inspect:

Bash

```bash
ros2 topic echo /submarine/estimated_fix
```

And watch the error logs from the evaluator terminal.

14) Where to put your real geo model

Inside `geo_estimator.py`, replace this function:

Python Run

```python
def run_geo_model(self, buoy_x, buoy_y, buoy_z, buoy_yaw, sub_x, sub_y, su
```

Right now it reconstructs the submarine location from the true relative geometry.
For your real work, change it so the estimator uses only the variables that your real system would know, for example:

- buoy RTK GPS
- buoy heading
- tether length
- tether angle / azimuth
- submarine depth

Example shape:

Python Run

```python
def run_geo_model(self, buoy_x, buoy_y, buoy_z, buoy_yaw, sub_x, sub_y, su
    # replace with your model inputs and math
    tether_length = ...
    tether_angle = ...
    depth = ...

    horizontal = ...
    est_dx = ...
    est_dy = ...
    est_dz = ...

    return buoy_x + est_dx, buoy_y + est_dy, buoy_z + est_dz
```

15) How to simulate tether measurements properly

Right now the code uses the submarine truth pose to derive geometry.
That is okay for **phase 1** because you are validating the math.

Next, add a dedicated logical tether node:

- input: buoy odom, submarine odom
- output:
  - `/tether/length`
  - `/tether/azimuth`
  - `/tether/elevation`

Then your estimator should subscribe to those topics instead of directly using submarine truth.

That gives you a cleaner research structure:

- Gazebo = ground truth generator
- Tether node = simulated onboard measurements
- Geo estimator = your actual algorithm
- Evaluator = accuracy checker

16) Add sensor noise

For more realistic experiments, add noise in your estimator inputs.

In `fake_buoy_gps.py`, add small Gaussian noise:

Python Run

```python
import random

lat_noise = random.gauss(0.0, 1e-7)
lon_noise = random.gauss(0.0, 1e-7)
alt_noise = random.gauss(0.0, 0.05)

nav.latitude = self.origin_lat + dlat + lat_noise
```

```
    nav.longitude = self.origin_lon + dlon + lon_noise
    nav.altitude = self.origin_alt + z + alt_noise
```

In the tether measurement stage, add noise to:

- tether length
- azimuth
- elevation
- depth

Then record error growth.

17) How to make it more realistic later

After this minimum version works, upgrade in this order:

Step A — floating buoy

Use Gazebo buoyancy so the buoy physically floats instead of staying static. Gazebo has buoyancy and hydrodynamics support for underwater / maritime simulation.

gazebosim.org +1

Step B — underwater submarine dynamics

Use the underwater vehicle tutorial and hydrodynamics-based model for the submarine. Gazebo's underwater vehicle support is based on Fossen-style equations and includes maritime thruster support. gazebosim.org +1

Step C — real tether physics

Do this only after your geo model works.
Options:

- segmented links and joints
- detachable / fixed attachment points
- custom drag model

For the first thesis version, **logical tether is enough**.

18) Optional: use `robot_localization`

If later you want a more standard GPS + IMU + odometry fusion chain, `robot_localization` provides `navsat_transform_node`, which converts GPS data into a world-consistent odometry frame using GPS, IMU heading, and odometry. docs.ros.org +2

For your current project, you do not need it yet because your own geo model is the main contribution.

19) Common problems

`gz sim` opens but no ROS topics

Make sure the bridge is running:

Bash

```bash
ros2 run ros_gz_bridge parameter_bridge /model/buoy/odometry@nav_msgs/msg/
```

`ros_gz_bridge` only bridges supported message pairs, and `nav_msgs/msg/Odometry <-> gz.msgs.Odometry` is supported. docs.ros.org

`set_pose` service not found

Your world must include:

XML

```xml
<plugin
  filename="gz-sim-user-commands-system"
  name="gz::sim::systems::UserCommands"/>
```

That plugin is what provides `/world/<world>/set_pose`. gazebosim.org +1

Wrong Gazebo / ROS version mix

Use a recommended pairing. ROS 2 Humble is recommended with Gazebo Fortress, and ROS 2 Jazzy with Gazebo Harmonic. gazebosim.org

20) Best experiment sequence for your FYP

Run these in order:

1. **Static buoy, static submarine**
   Verify your geo model.
2. **Static buoy, moving submarine**
   Move submarine using `set_pose`.
3. **Add measurement noise**
   RTK noise, tether noise, depth noise.
4. **Moving buoy, moving submarine**
   Later, after buoy physics is added.
5. **Hydrodynamic submarine**
   More realistic underwater motion.
6. **Tether sag / flexible cable**
   Final advanced stage.

21) What you should do next

Do this now, exactly in this order:

Bash

```bash
cd ~/fyp_ws/src
ros2 pkg create --build-type ament_python buoy_sub_geo_sim
```

Then add the files exactly as above, then:

Bash

```bash
cd ~/fyp_ws
colcon build
source install/setup.bash
ros2 launch buoy_sub_geo_sim sim.launch.py
```

Then move the submarine with:

Bash

```bash
gz service -s /world/ocean_world/set_pose \
  --reqtype gz.msgs.Pose \
  --reptype gz.msgs.Boolean \
```

```
    --timeout 1000 \
    --req 'name: "submarine", position: {x: 8, y: 2, z: -10}, orientation: {
```

After that, replace the example geometry inside `run_geo_model()` with your own FYP geo model.

I can next give you **version 2** with:

- moving buoy,
- tether measurement node,
- noise injection,
- CSV logging for thesis graphs.

Sources

GeoModel.ipynb
File

thi is geo model and how can i automamted simulatiom and collect simulated data into txt log file

Thought for 27s

Your notebook shows the core geo model idea, but there is one important issue:

In `calculate_coordinates()` you call:

Python Run

```python
results = cor(alpha, beta, mu, eta, L, z_input, sb, l0)
```

but your `cor()` function is defined as:

Python Run

```python
def cor(alpha,beta,mu,eta,L,z):
```